

Multimodal Autonomous Agent Application

Multimodal Autonomous Agent Application

- 1. Concept Introduction
 - 1.1 What is an "Autonomous Agent"?
 - 1.2 Implementation Principles Overview
- 2. Project Architecture
 - Key Code
 - 1. Agent Core Workflow (`largetmodel/utils/ai_agent.py`)
 - 2. Task Planning and LLM Interaction (`largetmodel/utils/ai_agent.py`)
 - 3. Parameter Processing and Data Flow Implementation (`largetmodel/utils/ai_agent.py`)
 - Code Analysis
- 3. Practical Operations
 - 3.1 Configure Online LLM
 - 3.2 Start and Test the Function

1. Concept Introduction

1.1 What is an "Autonomous Agent"?

In the `largetmodel` project, **Multimodal Autonomous Agent** is the most advanced form of intelligence. It is no longer simply responding to a single user command, but can **autonomously think, plan, and continuously call multiple tools to accomplish a complex goal**.

The core of this function is the `agent_call` tool or the **Tool Chain Manager** (`ToolChainManager`) behind it. When a user makes a complex request that cannot be completed through a single tool call, the autonomous agent starts.

1.2 Implementation Principles Overview

The autonomous agent implementation in `largetmodel` follows the industry mainstream **ReAct (Reason + Act)** paradigm. Its core idea is to imitate the human problem-solving process, cycling between "thinking" and "acting".

1. **Reason:** When the agent receives a complex goal, it first calls a powerful language model (LLM) to "think". It asks itself: "To accomplish this goal, what should my first step be? Which tool should I use?" The LLM's output is not the final answer but an action plan.
2. **Act:** The agent executes the corresponding action based on the LLM's thinking result—calling `ToolsManager` to run the specified tool (such as `visual_positioning`).
3. **Observe:** The agent obtains the result of the previous action ("observation"), for example, `{"result": "Found the cup, position at [120, 300, 180, 360]"}`.
4. **Think Again:** The agent submits this observation result, along with the original goal, to the LLM for the second round of "thinking". It asks itself: "I have found the cup's position. To know its color, what should I do next?" The LLM may generate a new action plan, for example, `{"thought": "I need to analyze the image area where the cup is located to determine the color", "action": "seewhat", "args": {"crop_area": [120, 300, 180, 360]}}`.

This **Reason -> Act -> Observe** cycle continues until the original goal is achieved, at which point the agent generates and outputs the final answer.

2. Project Architecture

Key Code

1. Agent Core Workflow (`largemodel/utils/ai_agent.py`)

The `_execute_agent_workflow` function is the main execution loop of the Agent, defining the core flow of "Plan -> Execute".

```
# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...

    def _execute_agent_workflow(self, task_description: str) -> Dict[str, Any]:
        """
        Executes the agent workflow: Plan -> Execute -> Reflect -> Adjust.
        """
        try:
            # Step 1: Task Planning
            self.node.get_logger().info("AI Agent starting task planning phase")
            plan_result = self._plan_task(task_description)
            self.node.get_logger().info(f"Task planning result: {plan_result}")

            if not plan_result["success"]:
                self.node.get_logger().error(f"Task planning failed: {plan_result.get('message', 'Unknown error')}")
                return plan_result

            self.task_steps = plan_result["steps"]
            self.node.get_logger().info(f"Task planning completed with {len(self.task_steps)} steps")

            # Send detailed planning results to the frontend
            if self.feedback_callback:
                plan_details = "\n".join([f"  - Step {s['id']}: {s['description']}" for s
in self.task_steps])
                self.feedback_callback(f"AI Agent task planning complete, {len(self.task_steps)} steps total:\n{plan_details}")

            # Step 2: Execute Task Steps
            execution_results = []
            step_outputs = {} # Store step outputs for subsequent steps
            tool_outputs = [] # Store outputs in ToolOutput format for toolchain data
            passing

            for i, step in enumerate(self.task_steps):
                step_info = f"Executing step {i+1}/{len(self.task_steps)}: {step['description']}"
```

```

self.node.get_logger().info(step_info)
if self.feedback_callback:
    self.feedback_callback(step_info)

# Skip steps without a tool
if not step.get("tool") or step.get("tool").strip() == "":
    self.node.get_logger().warn(f"Step {i+1} has no tool specified, skipping
execution")

    execution_results.append({
        "success": True,
        "step_id": step.get("id"),
        "description": step["description"],
        "message": "skipped (no tool)"
    })
    continue

# Step 1: Process parameters before execution
# This method finds {...} references and replaces them with actual outputs
from previous steps
try:
    processed_parameters =
self._process_step_parameters(step.get("parameters", {}), tool_outputs)
    step["parameters"] = processed_parameters # Update parameters in the
step

except Exception as e:
    self.node.get_logger().error(f"Parameter processing failed, step {i+1}
will be skipped: {e}")
    step_result = {
        "success": False,
        "step_id": step.get("id"),
        "description": step["description"],
        "error": f"Parameter processing failed: {e}",
        "message": f"Step execution failed: {e}"
    }
    execution_results.append(step_result)
    # If parameter processing fails, terminate the entire task
    return {"success": False, "message": f"Task terminated due to parameter
processing failure in step {i+1} ('{step['description']}'): {e}", "results":
execution_results}

# Record the step in the current task
step_record = {
    "step_id": step.get("id"),
    "description": step["description"],
    "start_time": datetime.now()
}

# Step 2: Execute the step with processed parameters
step_result = self._execute_step(step, tool_outputs)
execution_results.append(step_result)

# Step 3: Save the output for subsequent steps to reference
if step_result.get("success"):

```

```

        # Use the complete ToolOutput object returned by the tool adapter
        if step_result.get("tool_output"):
            tool_outputs.append(step_result["tool_output"])
            self.node.get_logger().info(f"Step {i+1} output added to the tool
chain passing list")
        else:
            # Also add failed steps to the tool output list, but mark them as failed
            if step_result.get("tool_output"):
                tool_outputs.append(step_result["tool_output"])
            else:
                from utils.tool_chain_manager import ToolOutput
                tool_output = ToolOutput(
                    tool_name=step.get("tool"),
                    success=False,
                    data=None,
                    metadata={"step_id": step.get("id"), "description":
step.get("description")},
                    error_message=step_result.get("error", "Step execution failed
and did not generate a standard output object")
                )
                tool_outputs.append(tool_output)

            # Complete the step record
            step_record["end_time"] = datetime.now()
            step_record["success"] = step_result["success"]
            step_record["result"] = step_result
            if self.current_task:
                self.current_task["steps"].append(step_record)

            # If a step fails, abort the entire task
            if not step_result.get("success", False):
                self.node.get_logger().error(f"Step {i+1} execution failed:
{step_result.get('message', 'Unknown error')}")
                self.node.get_logger().error(f"Task '{task_description}' terminated
early due to a failed step.")
                # Return a final result with failure information
                return {
                    "success": False,
                    "message": f"Task terminated because step '{step['description']}'
failed.",
                    "steps_executed": i + 1,
                    "results": execution_results
                }
            else:
                self.node.get_logger().info(f"Step {i+1} executed successfully")
                # Send successful tool execution result
                if self.feedback_callback:
                    tool_output = step_result.get("tool_output")
                    if tool_output:
                        feedback = self._format_tool_output_for_feedback(tool_output)
                        self.feedback_callback(feedback)

        summary = self._summarize_execution(task_description, execution_results)

```

```

# Check for generated documents
generated_files = []
for result in execution_results:
    if result.get("success") and "Saved to:" in str(result.get("result", "")):
        # Extract file path
        result_text = str(result.get("result", ""))
        if "Saved to:" in result_text:
            file_path = result_text.split("Saved to:")[1].strip()
            generated_files.append(file_path)

return {
    "success": True,
    "message": summary,
    "steps_executed": len(execution_results),
    "results": execution_results,
    "generated_files": generated_files
}

except Exception as e:
    self.node.get_logger().error(f"AI Agent workflow execution exception: {e}")
    import traceback
    self.node.get_logger().error(f"Exception stack: {traceback.format_exc()}")
    return {"success": False, "message": f"Workflow execution failed: {str(e)}"}

```

2. Task Planning and LLM Interaction (`largemodel/utils/ai_agent.py`)

The core of the `_plan_task` function is constructing a sophisticated Prompt that leverages the reasoning capability of the large model itself to generate a structured execution plan.

```

# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _plan_task(self, task_description: str) -> Dict[str, Any]:
        """
        Uses the large model for task planning and decomposition.
        """
        try:
            self.node.get_logger().info("Starting to construct planning prompt")

            tool_descriptions = []
            for name, adapter in self.tools_manager.tool_chain_manager.tools.items():

                description = adapter.input_schema.get("description", "No description
available.")

                # Extract parameter names for prompts
                props = adapter.input_schema.get("properties", {})
                params = ", ".join(props.keys())

```

```

        tool_descriptions.append(f"- {name}({params}): {description}")

    available_tools_str = "\n".join(tool_descriptions)

    planning_prompt = f"""
    As a professional task planning Agent, please decompose the user task into a
    series of specific, executable JSON steps.

    *** Available Tools:***
    {available_tools_str}
    *** Core Rules:***
    1.  **Data Passing**: When subsequent steps need to use the output from previous
    steps, you must use the format `{{{steps.N.outputs.KEY}}}` for reference.
        - `N` is the step ID (starting from 1).
        - `KEY` is the specific field name in the previous step's output data.
        - After `outputs`, it can be `data` (main data) or `metadata.sub_key`
    (metadata).
    2.  **JSON Format**: You must strictly return a JSON object, without any
    markdown wrapping (like ```json```).
    3.  **Tool Selection**: Strictly select the most appropriate tool based on the
    tool description.

```

3. Parameter Processing and Data Flow Implementation (`largemodel/utils/ai_agent.py`)

The `_process_step_parameters` function is responsible for parsing placeholders and implementing data flow between steps.

```

# From largemodel/utils/ai_agent.py

class AIAgent:
    # ...
    def _process_step_parameters(self, parameters: Dict[str, Any], previous_outputs:
List[Any]) -> Dict[str, Any]:
        """
        Parses parameter dictionary, finds and replaces all {...} references using regex.
        This implementation can handle multiple references within a single string.
        """
        processed_params = parameters.copy()
        pattern = re.compile(r"\{\{steps\.(?>0\d+)\.outputs\.(.+?)\}\}")

        for key, value in processed_params.items():
            if not isinstance(value, str):
                continue

            # Use an inner function as the replacement argument for re.sub to execute find
            logic for each match
            def replacer(match):
                step_id = int(match.group(1))

```

```

        path = match.group(2).strip() # .strip() ensures the path has no extra
spaces

        self.node.get_logger().info(f"Found reference in parameter '{key}': step_id={
step_id}, path='{path}'")

        if not (1 <= step_id <= len(previous_outputs)):
            raise ValueError(f"Invalid step ID referenced: {step_id} (available
steps: {len(previous_outputs)})")

        # Get the corresponding step output
        target_output = previous_outputs[step_id - 1]

        # Extract the real value from the ToolOutput object based on the path
        retrieved_value = self._get_value_from_path(target_output, path)

        if retrieved_value is None:
            available_keys = list(target_output.to_dict().keys())
            available_metadata_keys = list(target_output.to_dict().get('metadata',
{}).keys())

            raise ValueError(f"Path '{path}' not found in the output of step
{step_id}. Available top-level keys: {available_keys}. Available metadata keys:
{available_metadata_keys}")

        # The returned value must be a string to be embedded in the original
parameter string
        return str(retrieved_value)

    try:
        # Use re.sub and our replacer function to replace all found placeholders
        # Check if the placeholder pattern exists in the original value
        if pattern.search(value):
            new_value = pattern.sub(replacer, value)

            single_match = pattern.fullmatch(value)
            if single_match:
                step_id = int(single_match.group(1))
                path = single_match.group(2).strip()
                retrieved_value = self._get_value_from_path(previous_outputs[step_id
- 1], path)

                if retrieved_value is not None:
                    processed_params[key] = retrieved_value
                    self.node.get_logger().info(f"Parameter '{key}' has been
directly replaced with the value (type: {type(retrieved_value).__name__})")
                else:
                    raise ValueError(f"Path '{path}' not found in the output of step
{step_id}")
            else:
                processed_params[key] = new_value
                self.node.get_logger().info(f"Parameter '{key}' updated. Old value:
'{value}', New value: '{new_value}'")

    except ValueError as e:

```

```

        raise e

    return processed_params

def _get_value_from_path(self, output_obj, path: str) -> Any:
    parts = path.split('.')
    current_value = output_obj.to_dict() # Start with the full object as a dictionary

    for part in parts:
        match = re.match(r"(\w+)\[(\d+)\]", part)

        if match:
            key = match.group(1)
            index = int(match.group(2))

            # First, get the list using the key
            if isinstance(current_value, dict) and key in current_value:
                target_list = current_value.get(key)

                # Then, access the element using the index
                if isinstance(target_list, list):
                    if 0 <= index < len(target_list):
                        current_value = target_list[index]
                    else:
                        self.node.get_logger().error(f"Path parsing error: index {index}
is out of range for list '{key}' (size: {len(target_list)})")
                        return None # Index out of bounds
                else:
                    self.node.get_logger().error(f"Path parsing error: '{key}' is not a
list and cannot be accessed by index.")
                    return None # Trying to index a non-list
            else:
                self.node.get_logger().error(f"Path parsing error: key '{key}' not found
in the current data.")
                return None # Key not found
        else:
            # Fallback to simple dictionary key access
            if isinstance(current_value, dict) and part in current_value:
                current_value = current_value[part]
            else:
                available_keys = list(current_value.keys()) if isinstance(current_value,
dict) else []
                self.node.get_logger().error(f"Path parsing error: key '{part}' not
found in the current data. Available keys: {available_keys}")
                return None # Path is invalid

    return current_value

```


Code Analysis

The AI Agent is the "central brain" of the system. It transforms advanced, and sometimes even vague, user tasks into a series of precise, ordered tool calls. Its implementation does not rely on any specific model platform but is built on a universal, scalable architecture.

1. **Dynamic Task Planning:** The core capability of the Agent lies in the `_plan_task` function. It does not rely on hard-coded logic but dynamically generates task plans by interacting with the large model.
 - **Self-awareness and Prompt Construction:** At the beginning of planning, the Agent first checks all currently available tools and their descriptions. Then, it packages this tool information, the user task, and strict rules (such as data transfer format) into a highly structured `planning_prompt`.
 - **Model as Planner:** This Prompt is sent to a general-purpose text large model. The model reasons based on the provided context and returns a JSON-formatted action plan containing multiple steps. This design is highly scalable: when a tool is added or modified in the system, the Agent's planning capability automatically updates without needing to modify the Agent's own code.
2. **Tool Chain and Data Flow:** Real-world tasks often require multiple tools to collaborate. For example, "take a photo and describe" requires the output of the "take photo" tool (image path) as the input for the "describe" tool. The AI Agent elegantly achieves this through the `_process_step_parameters` function.
 - **Data Reference Placeholders:** During the planning phase, the large model embeds special placeholders in parameter values that need to pass data, such as `{{steps.1.outputs.data}}`.
 - **Real-time Parameter Replacement:** In the main loop of `_execute_agent_workflow`, before executing each step, `_process_step_parameters` is called. It uses regular expressions to scan all parameters of the current step. Once a placeholder is found, it finds the corresponding data from the previous step's output list and performs real-time replacement. This mechanism is the key to achieving complex task automation.
3. **Supervisory Execution and Fault Tolerance:** `_execute_agent_workflow` forms the Agent's main execution loop. It strictly follows the planned step order, executing each action in sequence, and ensures data is correctly passed between steps.
 - **Atomic Steps:** Each step is treated as an independent "atomic operation". If any step fails to execute, the entire task chain immediately stops and reports an error. This ensures the system's stability and predictability, avoiding continued execution in an erroneous state.

In summary, the universal implementation of AI Agent demonstrates an advanced software architecture: it does not directly solve problems but builds a framework that allows an external, general-purpose reasoning engine (large model) to solve problems. Through the two core mechanisms of "dynamic planning" and "data flow management", the Agent can orchestrate a series of independent tools into complex workflows that can accomplish advanced tasks.

3. Practical Operations

3.1 Configure Online LLM

1. Update the key in the configuration file, open the model interface configuration file

`large_model_interface.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/large_model_interface.yaml
```

2. Fill in your API Key:

Find the corresponding section and paste the API Key you just copied. Here taking Qwen configuration as an example

```
# large_model_interface.yaml

## Qwen
qianwen_api_key: "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx" # Paste your Key
qianwen_model: "qwen-vl-max-latest" # You can choose models as needed, such as qwen-
turbo, qwen-plus
```

3. Open the main configuration file `yahboom.yaml`:

```
vim ~/yahboomcar_ws/src/largemodel/config/yahboom.yaml
```

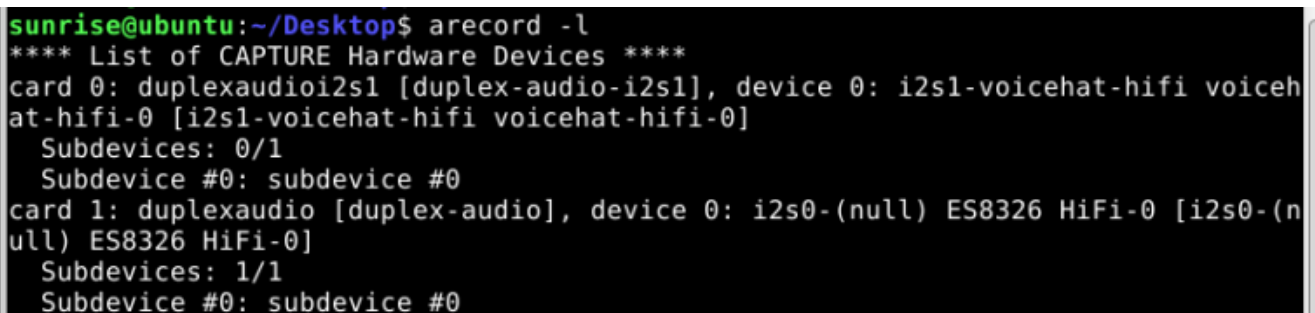
4. Select the online platform to use:

Modify the `llm_platform` parameter to the platform name you want to use

```
# yahboom.yaml

model_service:
  ros__parameters:
    # ...
    llm_platform: 'tongyi' # Optional platforms: 'tongyi', 'spark', 'qianfan',
'openrouter'
```

5. When using the voice module for broadcast, check if a device is occupying the voice module. You can first enter



```
sunrise@ubuntu:~/Desktop$ arecord -l
**** List of CAPTURE Hardware Devices ****
card 0: duplexaudioi2s1 [duplex-audio-i2s1], device 0: i2s1-voicehat-hifi voicehat-hifi-0 [i2s1-voicehat-hifi voicehat-hifi-0]
Subdevices: 0/1
Subdevice #0: subdevice #0
card 1: duplexaudio [duplex-audio], device 0: i2s0-(null) ES8326 HiFi-0 [i2s0-(null) ES8326 HiFi-0]
Subdevices: 1/1
Subdevice #0: subdevice #0
```

If the output is as shown in the image above, you can directly use the broadcast device number without modification. If different, open the configuration file

```
vim ~/yahboomcar_ws/src/largemodel/largemodel/model_service.py
```

Find the location in the image and modify it to plughw:1,0 or hw:1,0, then save.

```
172         except Exception as e:
173             self.get_logger().error(f"❌ Error in TTS callback: {e}")
174             # 出错时恢复麦克风
175             self.publish_wakeup_signal(False)
176
177     def play_tts_audio(self):
178         """播放TTS音频文件"""
179         try:
180             if not os.path.exists(self.tts_out_path):
181                 self.get_logger().error(f"❌ Audio file not found: {self.tts_out_path}")
182                 return
183
184             # 使用aplay命令播放
185             self.get_logger().info(f"🔊 Playing audio: aplay -D plughw:0,0 {self.tts_out_path}")
186
187             result = subprocess.run(
188                 ["mpg123", "-a", "plughw:1,0", self.tts_out_path],
189                 capture_output=True,
190                 text=False
191             )
192
193             if result.returncode == 0:
194                 self.get_logger().info("✅ TTS playback completed successfully")
195             else:
196                 self.get_logger().error(f"❌ TTS playback failed: {result.stderr}")
197
198         except Exception as e:
199             self.get_logger().error(f"❌ Error playing TTS audio: {e}")
200
```

After modifying the configuration file, you need to recompile in the workspace and source:

```
cd ~/yahboomcar_ws
colcon build && source install/setup.bash
```

3.2 Start and Test the Function

1. Open a new terminal and start the `audio_io` main program:

Run the following command to enable the microphone and wake-up program:

```
ros2 launch audio_io audio_io_yahboom.launch.py
```

2. Open a new terminal and start the `largemodel` main program:

Run the following command to enable voice interaction:

```
ros2 launch largemodel largemodel_control.launch.py
```

3. Test:

- **Wake-up:** Say to the microphone: "Hello, Digua."
- **Dialogue:** After the speaker responds, you can say: `Based on the current environment, generate an image similar to the scene`

- **Observe logs:** In the terminal running the `launch` file, you should be able to see:

The system displays that it received the text instruction, calls the `aiagent` tool, and then provides the prompt to the LLM. The LLM will analyze the detailed steps for calling the tool. For example, with this question, it will call the `seewhat` tool to get the image, then provide the image to the LLM for parsing. The parsed text will be sent back to the LLM as the content for the new image to generate.

- **Listen to the answer:** After a while, you should hear the completion response from the speaker, and the camera view and the newly generated image will pop up. Later, you can find the newly generated image in the path `~/yahboomcar_ws/src/largemodel/resources_file/generated_images`.